You can then use a *compiler directive* to configure a different URL depending on whether the app was run from Xcode or downloaded via TestFlight. By default, running the app via Product → Run puts it in debug mode so you can attach a debugger and step through code. And archiving a build via Product → Archive is set to release mode, where `#if DEBUG` returns `false`.

Update `SceneDelegate` to set a different `baseURL` depending on whether the app was built in debug mode or not.

ch09_04/ios/App/Delegates/SceneDelegate.swift

```
    import HotwireNative
    import UIKit

    #if DEBUG
    let baseURL = URL(string: "http://localhost:3000")!
    #else
»   // NOTE: Your IP address will be different.
»   let baseURL = URL(string: "http://192.168.50.139:3000")!
    #endif

    class SceneDelegate: UIResponder, UIWindowSceneDelegate {
        // ...
    }
```

Pop back into your terminal and stop the Rails server. Then run it again via `bin/rails server -b 0.0.0.0`. The `-b` option binds Rails to a specific IP address. Using `0.0.0.0` binds it to *all* IP addresses, not just `localhost`.

Back in Xcode, archive and upload a new build one more time. When it finishes processing, download it on your iOS device. Don't forget to declare encryption if you didn't update `Info.plist`!

Finally, launch the app that now correctly hits the server running on your Mac. You've got an app running on a real device—go show your friends!